

SENTINEL

Sensor Network for Tracking, Integration, and Event Logging
A Canadian Armed Forces surveillance operations centre

Lab 1, Part 1

The operating system interface
`libsyscall`

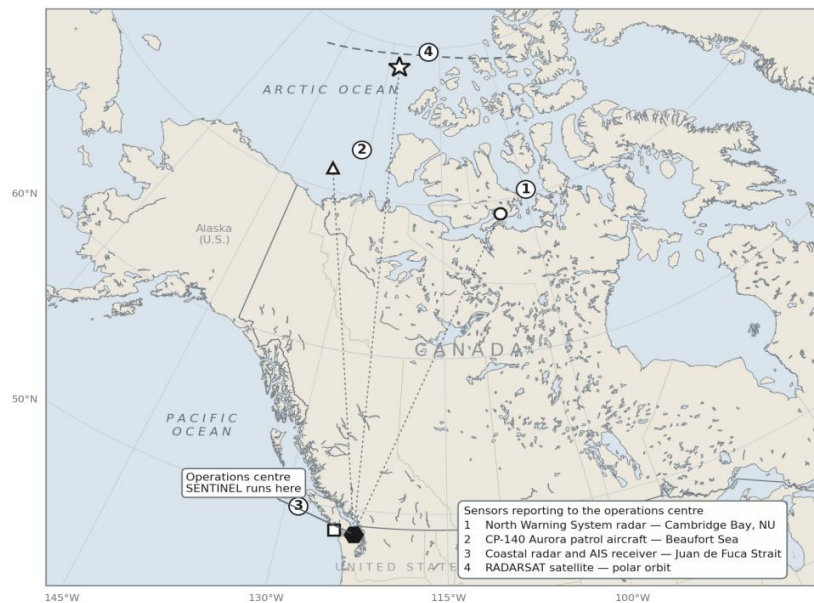


Figure 1. The four sensors of the detachment, and the operations centre they report to. Positions are approximate.

Watch serial 2026-282

Detachment	SENTINEL, northern and Pacific approaches
Sensors	four, reporting continuously
Centre	records every report. It does not yet resolve identity
Carried over	one contact in the strait broadcasts an identity that is filed nowhere

Lab session	Friday, October 9, 2026
Prelab due	Wednesday, October 7, 2026, by midnight (Brightspace)
Deliverable due	Friday, October 23, 2026 (Brightspace)

Lab 1 total	100 marks, which is 5% of the course grade
This deliverable	80 marks

Figures 1 and 2 are from the ECE 476 SENTINEL lab documentation.

Contents

1	Your posting	4
2	SENTINEL, and what Part 1 adds	5
3	Three operational cases	7
4	Before you start	10
5	The repository, and what you are given	11
6	The operations centre display	14
7	What you build	15
8	Assessment	28
9	Submission	29

1 Your posting

You are posted as a software engineer to a surveillance detachment. The detachment covers Canada's northern and Pacific approaches. It operates four sensors and one operations centre. A watch officer sits at the display in that centre for the length of a watch, and acts on what the display shows.

Figure 1 shows where the four sensors are. Table 1 states what each one gives you.

Table 1: The four sensors of the detachment, and what each one contributes.

	Sensor	Location	What it reports	Rate	Strength
	North Warning System radar	Cambridge Bay, NU	aircraft position, altitude, speed	20/s	constant watch
	CP-140 Aurora patrol aircraft	Beaufort Sea	contacts it finds and identifies up close	5/s	identifies
	Coastal radar and AIS receiver	Juan de Fuca Strait	vessel positions, and the identity each ship broadcasts	50/s	names ships
	RADARSAT satellite	polar orbit	a full contact list for a wide area, once per pass	2000 per burst	wide area

No single sensor is enough. The radar watches all the time but cannot tell you what a contact is. The Aurora can identify a contact but can only be in one place. AIS names the ships that choose to broadcast, so a radar contact with no AIS is exactly the kind of thing a watch officer wants to know about. The satellite covers everything, but only during a pass. Software that combines them is what makes the four useful together.

The four areas overlap. RADARSAT's pass covers the whole of the other three, and the radar at Cambridge Bay and the Aurora over the Beaufort watch a band of sea and air in common. One aircraft in that band is therefore reported by two sensors at once, and each report gives a slightly different position, because each sensor measures rather than knows. Making one account out of several is the work of the centre. It is not work any one sensor can do.

1.1 The state of the current watch

The detachment holds a watch of its own while you build this. The sensors report, the centre records, and one item is open.

A vessel in the Juan de Fuca Strait broadcasts an identity. That identity appears on no filed list the detachment holds. It may be a registration entered wrongly, a transmitter that was never registered, or a vessel that does not want to be known. **The centre cannot tell which, and Lab 1 does not give it the means to.**

What Lab 1 gives the centre is the record. Every report about that vessel, with a reliable time on it and nothing lost. Lab 3 is where the centre compares a report against the filed traffic and raises the alert. It can only do that against a record that is complete, and making the record complete is this lab.

1.2 The rule that governs the system

Machines watch. People decide.

This is the rule in Canadian and NORAD practice, and it decides what your software is allowed to do. Software collects the reports, records them, and sorts them by importance. A human decides what to do about them. Sending fighters from Cold Lake or Bagotville to look at an unidentified aircraft is always a human decision.

Your software therefore has one duty above all others: **give the watch officer a true account, in time to act on it.** A report the centre did not read cannot be acted on. A report the centre did not record cannot be reviewed afterwards. A report with no reliable time on it cannot be placed in a sequence of events.

1.3 Why this lab is first

Part 1 is the layer that touches the operating system. Everything above it reads its input and writes its history through the library you build here.

The centre cannot decide anything about a report it did not read. It cannot prove anything it did not record. It cannot order anything it cannot timestamp. Those three sentences are your lab. Nothing built in Lab 2, Lab 3 or Lab 4 can be correct if this layer is wrong, because every one of them calls it.

1.4 Systems like this in service

SENTINEL is a teaching system, but its shape is not invented. Sensors reporting to a collection point, a record that has to be exact, and software that decides what matters first all appear in systems working today.

Marine AIS collects position and identity broadcasts from vessels at coastal receivers. **Civil air surveillance** combines radar returns with position reports broadcast by aircraft, which arrive at different rates and accuracies, into one track for each aircraft.

BRNKL, by Barnacle Systems of Victoria, is a monitoring and recording unit for working vessels. It is installed on patrol boats, coast guard craft, naval craft, fishing vessels and commercial fleets in more than forty countries. One unit collects data from the engine and power systems over NMEA 2000, from ModBus and CAN bus devices, from serial and Ethernet equipment, from cameras, and from body worn sensors. It time synchronises all of it, watches it with an alerting engine, and stores it locally. Afterwards an entire trip can be replayed on a time aligned display.

Read that list again and note what has to be true for any of it to work: every reading must carry a trustworthy time, and nothing may be lost between the device and the record. That is the library you write in this lab.

Both founders of Barnacle Systems are graduates of this department.

2 SENTINEL, and what Part 1 adds

SENTINEL is four C libraries. Each one is a layer on the one before it, and each lab builds one layer.

Part	Lab session	Library	What it adds to SENTINEL
Part 1	Oct 9	libsyscall	talking to the operating system: reading feeds, writing the mission log, precise timestamps
Part 2	Oct 30	libproc	one process per sensor, and the channels that bring their reports to the centre
Part 3	Nov 20	libipc	worker threads inside the centre, and the track picture
Part 4	Dec 4	libsync	protection for everything the workers share

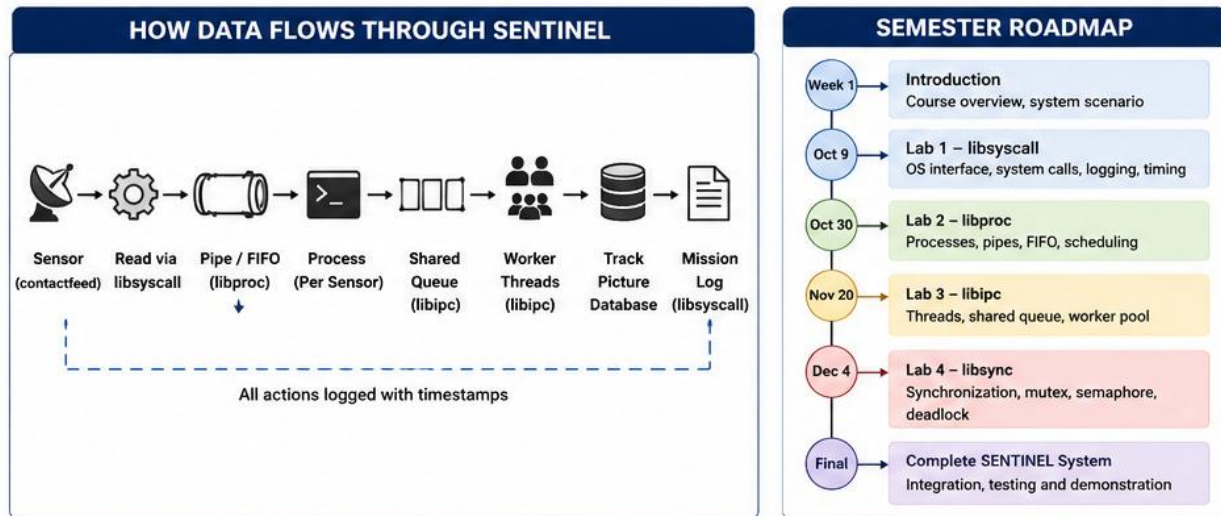


Figure 2. How data flows through the complete SENTINEL system. Lab 1 builds the two boxes marked *libsyscall*: the reading of the feed on the left, and the mission log on the right.

2.1 What Lab 1 runs

In the complete system, four sensor processes send reports to the centre. Those processes arrive in Lab 2. In Lab 1 there is one program, and it reads a file.

```

contactfeed  writes contact reports
↓
build/live/feed.txt  a file of report lines
↓  your demo_lab1, using your libsyscall
build/live/missions.log  the mission log
↓
make watch  serves the operations centre display
↓
a browser at http://localhost:8080/

```

`contactfeed` is a program the course gives you. It stands in for the four sensors, so that you can build and test the centre without them.

2.2 The requirement Lab 1 satisfies

SENTINEL has six requirements. They are the problems any real operations centre has, and each lab solves one group of them.

ID	Requirement	Solved in
R1	Four sensors run at very different rates, and any one can stop with no warning	Part 2
R2	An urgent contact must overtake routine traffic	Part 2
R3	Thousands of contacts can arrive in one burst	Part 3
R4	Many workers write the same shared record, and must not corrupt it	Part 4
R5	The centre must never freeze while it still looks alive	Part 4
R6	Every action is timestamped and written to the mission log	Part 1

R6 is your requirement. The mission log is the record of what the centre knew and when it knew it. It is written while events happen and read afterwards, by people who were not there. A log with a gap in it cannot answer a question about the gap. A log with an unreliable clock cannot put two events in order. Section 3 gives the case where that matters.

2.3 What the centre can do, and what it cannot do yet

Each lab adds one capability to the same centre. Knowing which ones are missing tells you what your own layer is responsible for, and what it is not.

The centre can	From	Still missing after Lab 1
Read a feed without losing any of it	Lab 1	
Put a real date on every report	Lab 1	
Record every report, in order	Lab 1	
	Lab 2	Run each sensor separately, and carry on when one stops
	Lab 2	Let an urgent report overtake routine traffic
	Lab 3	Absorb a burst of thousands
	Lab 3	Decide that two reports are one object, and compare it against the filed traffic
	Lab 4	Keep that picture exact while many workers change it

So the vessel in the strait stays unresolved at the end of this lab, and that is the correct outcome.

A centre that cannot yet identify a contact must still record it exactly. Every capability above is built on the record this lab produces, and none of them can repair a record that arrived incomplete.

3 Three operational cases

These are the three operations your library takes part in. Read them before you read what you have to build. Each one explains why a function in Section 7 exists, and what it costs the watch officer when that function is wrong.

Each case names the functions that carry it out. Those functions are specified in Section 7, and you are not expected to recognise them yet.

Case 1. Normal operations

1. The watch officer starts the operations centre at the beginning of a watch.
2. The four sensors report. The radar sends about 20 reports each second, the Aurora about 5, the coastal receiver about 50.
3. Every report is read, given a time, and written to the mission log.
4. The display shows a routine watch. Reports arrive, and nothing is outstanding.

Your code: `sc_open_read` opens the feed, `lr_next_line` returns whole lines from it, `contact_parse` turns each line into a record, `sc_stamp` puts the time on it, and `log_writelf` records it.

Two ways to run this case

Your library does the same work in both, and the difference is whether the sensors are still reporting while you read.

A recorded feed. `make lab1` writes a file of reports first, and starts your program after the file is complete. The four sensors write one after another and as fast as they can, so the whole file covers a few milliseconds and the rates above do not apply. Your program reads to the end of the file and stops.

This is a replay. A detachment replays a recorded feed to answer a question about a watch that has finished, which is Case 3. It is also the quickest way to run your code while you build it, and it is what the acceptance test in Section 7 uses.

A live watch. The sensors report at the rates in the list above, and your program reads the file while they are still writing to it. The file never ends, so your program must wait for more and decide for itself when the watch is over.

That is what `--follow` does, and it is the harder of the two. Section 7.6 states the rules your loop must obey and gives the three commands that start a live watch.

Run it: `make lab1` for the recorded feed. For the live watch, see Section 7.6.

Case 2. A contact that is not on any list

1. The coastal receiver in the Juan de Fuca Strait reports a vessel making about 5 knots. It broadcasts an identity, as vessels in the lane do.
2. That identity is on no list the detachment holds. Every other vessel in the lane matches a filed registration. This one matches nothing.
3. Your library reads the report while routine traffic arrives from the other three sensors, gives it a time, and writes it to the mission log with everything else.
4. RADARSAT passes over the same water and reports the vessel again, from a position a few kilometres from the coastal receiver's. Both reports are recorded.

Your code: the same path as Case 1. Nothing in this lab treats the vessel differently, and nothing should.

What Lab 1 does not do yet. It does not compare the identity against the filed traffic, so it does not know the vessel is unmatched, and it raises no alert. It also does not know that the two reports, from two sensors, are one vessel. Deciding both of those is Lab 3's work.

What Lab 1 decides is whether the question can be answered at all. An alert in Lab 3 is raised by comparing a record against a list. If this lab drops the reports, mistimes them, or stops reading the channel after one bad line, then the vessel was never there as far as the centre is concerned, and no later lab can recover it.

Run it. make `lab1`, then make `watch`, and look at the Mission log view. The strait traffic reports under identities beginning `VESL`. One of them is the vessel in this case, and nothing on the screen marks it out, which is the point.

Case 3. The record is questioned

1. Some time after the watch, the detachment is asked what the centre knew about the vessel in Case 2, and when it knew it.
2. Nobody who was on watch is available, and no screen is still showing that day.
3. The answer comes from the mission log alone.

This case is why three decisions in Section 7 are what they are:

- The timestamp uses `CLOCK_REALTIME`, because the answer must be a real date and time, not a count of seconds since the program started.
- The log is opened with `O_APPEND`, because in Lab 2 several processes write it, and a record with lines missing cannot answer a question about the missing lines.
- The log is created `0600`, because it is a surveillance record.

Your code: `log_open`, `sc_stamp`, `log_writetf`, `log_close`.

3.1 The priority flag

The last field of a contact report is the priority flag. It is one letter. `P` means the report is urgent. `R` means it is routine.

The sensor sets the flag, and the centre obeys it. A sensor marks a report urgent because of something it observed. An aircraft does not answer a radio call. A transponder sends an emergency code. A vessel stops in a traffic lane. An aircraft turns towards protected airspace. The sensor cannot decide what to do about any of these, and it does not try. It marks the report, and the report carries that mark to the centre.

The flag belongs to the report, not to the contact. One aircraft sends many reports during a watch. Some of them can be urgent and the others routine, because the reason to be urgent starts and stops. A contact is therefore not "a priority contact". Its latest report is urgent, or it is not.

The feed you are given sets the flag on one report in ten. It does not choose which contact. The course supplies the feed in place of four sensors, and a feed that marked the contacts worth

looking at would be answering the question the centre exists to answer. Your library must treat every report the same way, whatever the flag says.

Part	What it does with the flag
Lab 1	Records it. A report with P is written to the mission log at level <code>PRIORITY</code> , and a report with R at level <code>INFO</code> . The order does not change
Lab 2	Acts on it. An urgent report overtakes routine traffic, which is R2. Lab 2 also adds a second channel that carries urgent reports from outside the four sensors

Lab 1 does not make an urgent report overtake anything. It records reports in the order they arrive. That is the correct behaviour for this lab. An operations centre cannot serve an urgent report before a routine one until it can hold both at once and choose, and holding both at once is Lab 2.

4 Before you start

4.1 The machine

Follow the Setup Guide to set your machine up for this lab. The Setup Guide, **Setting up the machine you build on**, is on Brightspace beside this manual. After you clone the repository, it is also in `docs/setup/setup.pdf`. It has a section for a Mac, for Windows and for Linux. It then installs the tools and checks that a build works. The rest of this section states what the guide sets up.

You need a Linux system on an `aarch64` or `x86_64` processor. A virtual machine is suitable. A Mac with Apple silicon runs `aarch64` Linux, and most PCs and WSL2 run `x86_64` Linux. This command reports which one you have.

```
uname -m
```

On Windows, keep your repository in the Linux home directory and not on the Windows drive. Your work belongs in `~`, not under `/mnt/c`. Lab 1 creates the mission log with permissions `0600`, and a Windows drive cannot store Unix permissions. The acceptance test checks those permissions exactly, so a repository in the wrong place fails checks that your code has passed.

4.2 The tools

```
sudo apt install build-essential python3 valgrind git zip procps
```

Tool	What the lab uses it for
<code>gcc, make</code>	compile and link your library, your tests and your demo
<code>python3</code>	serve the operations centre display to your browser
<code>valgrind</code>	find memory faults and leaks in your code
<code>git</code>	receive this lab, and the three labs after it
<code>zip</code>	build the file you submit
<code>pgrep, watch</code>	find your running demo, and read its <code>/proc</code> files (<code>procps</code>)

If a manual page named in the prelab is missing, install the manual pages as well:

```
sudo apt install manpages-dev
```

4.3 Your copy of the repository

You do not work in the course repository. You make one private copy of it, and you keep that copy for all four labs. The course announces the address of the course repository.

1. Make your copy. On GitHub use **Use this template**, then set the new repository to **Private**. On GitLab use **Fork**, then set the visibility to **Private**.

On GitHub, do **not** press Fork. A fork of a public repository is public, and GitHub cannot make a fork private. Your solution must not be public.

2. Clone your copy, and point it at the course:

```
git clone <your repository>
cd sentinel-lab
git remote add upstream <the course repository>
```

3. Each lab arrives with one command, when the course releases it:

```
git pull upstream main
```

You do not hand in this repository. You hand in a zip file, and Section 9 says how.

4.4 The first build

From the top of your copy:

```
make
```

Before you write anything, it ends with these lines, and they are correct:

```
src/libsyscall: no Makefile yet
src/libproc: no Makefile yet
src/libipc: no Makefile yet
src/libsync: no Makefile yet
src/sentinel: no Makefile yet
```

`make` builds the given code, copies the given programs into `build/`, and then builds each lab that has a `Makefile`. No lab has one yet, so it names each of the five and finishes with no error. Your first task in Section 7.7 is that `Makefile`.

5 The repository, and what you are given

Everything in the repository is either given to you or written by you. Nothing is hidden, and every file named in this manual is already on your disk after `git clone`.

sentinel-lab/

Makefile

README.md

common/

contact.h contact.c

linereader.h linereader.c

the top-level build. GIVEN

the repository overview. GIVEN

shared and given code

the contact record, and `contact_parse`

the line reader

```

cpu.h  cpu.c                the processor instruments
device.h device.c          opening a serial device
traffic.h traffic.c  traffic.txt  the traffic the centre expects
docs/
  DEFECTS.md              your record of faults you met
  setup/setup.pdf        the Setup Guide. GIVEN
  lab1/lab1_manual.pdf   this manual. GIVEN
  lab1/prelab1.pdf       the prelab. GIVEN
  lab1/                  YOUR three files of one run go here
src/libsyscall/          *** LAB 1 IS THIS DIRECTORY ***
  README.md              the quick reference
  include/sentinel/syscall.h  the interface. GIVEN. Do not change it
  Makefile               YOURS. It does not exist yet
  src/                   YOURS. Empty. Four files go here
  tests/                 YOURS. Empty. Your tests and your demo
src/libproc/  src/libipc/  src/libsync/  src/sentinel/
Labs 2, 3 and 4. Empty for now
tools/bin/aarch64/  tools/bin/x86_64/  the given programs, per processor
web/                the operations centre display. GIVEN

```

5.1 Where each given file is

What	Where	Owner
The interface you implement	src/libsyscall/ include/sentinel/syscall.h	given
The contact record and its parser	common/contact.h	given
The line reader	common/linereader.h	given
The processor instruments	common/cpu.h	given
Opening a serial device	common/device.h	given
The feed generator	tools/bin/<arch>/contactfeed. make copies it to build/contactfeed, which is the one you run	given
The acceptance test	tools/bin/<arch>/accept_lab1.o	given
The display	web/	given
This manual	docs/lab1/lab1_manual.pdf	given
The prelab	docs/lab1/prelab1.pdf	given
The Setup Guide	docs/setup/setup.pdf	given
Your library	src/libsyscall/src/io.c, clock.c, log.c, proc.c	yours
Your tests	src/libsyscall/tests/test_*.c	yours
Your demo	src/libsyscall/tests/demo_lab1.c	yours
Your build file	src/libsyscall/Makefile	yours

src/libsyscall/src/ and src/libsyscall/tests/ are empty. Those two directories are the lab. Everything else in the list above is already written.

build/ is not in the repository. `make` creates it, and everything `make` produces goes there. A fresh copy of the repository has no `build/` directory at all, and none of the commands in this manual work until you have run `make` once.

`contactfeed` stands in for the four sensors. The repository holds one program for each processor, in `tools/bin/aarch64/` and `tools/bin/x86_64/`. `make` copies the one your machine needs to `build/contactfeed`, and that copy is what every command in this manual runs.

`contactfeed` has a **profile**, which chooses what kind of traffic it sends. A profile changes the sky, and never the format of a report. Section 7.9 states which profile to run, and when.

5.2 The interface file

`src/libsyscall/include/sentinel/syscall.h` is the contract between your library and the rest of SENTINEL. Open it now and read all of it. It declares the fifteen functions you write, and states for each one what it must do, what it returns, and how it reports an error.

Do not change a declaration in it. Tests that are not in your repository call your functions exactly as that file declares them. If you change a declaration, those tests no longer compile against your library, and the work cannot be marked.

5.3 Code you may call but do not write

The headers in `common/` are given because no requirement of SENTINEL is about them. Lab 1 uses the four in the table. The fifth, `common/traffic.h`, is for a later lab. They carry data in and out of the system, or they measure a run. You may call anything they declare.

Header	What it gives you
<code>common/contact.h</code>	<code>contact_parse</code> , which turns one line of text into a record. <code>contact_strerror</code> , which names the reason a line was refused. <code>sensor_name</code> , which gives the three letters of a sensor
<code>common/linereader.h</code>	<code>lr_open</code> , <code>lr_open_follow</code> , <code>lr_next_line</code> , <code>lr_close</code>
<code>common/cpu.h</code>	which processors a program may use, and how much processor time a thread used
<code>common/device.h</code>	<code>device_open</code> , for a serial port

Each header states the contract of each function it declares: what the function needs, what it returns, and what it does on an error. Read `common/contact.h` before you call `contact_parse`. It returns `CONTACT_OK` for a report, and otherwise the reason the line is not one, and it changes the record only when the line is a report.

Read `common/linereader.c` before you write Section 7.6. It is given, but your demo is its caller, and the caller is where the difficult part is. Its comments say why each of its three answers exists.

6 The operations centre display

The repository includes a display. It is in `web/`, it is given, and it shows the run that your own programs produce.

No mark depends on it. It is a read-only view: it reads the files your programs write, and it changes nothing. It is there so that you can see what your code did.

6.1 How to start it

You need two terminals, both at the top of your repository.

In the first, produce a run:

```
make lab1
```

In the second, serve the display:

```
make watch
```

Then open `http://localhost:8080/` in a browser. Press Ctrl-C in the second terminal to stop the server.

`make watch` prints what it is doing, and you should see this:

```
SENTINEL page
-----
open      http://localhost:8080/
run data  /home/you/sentinel-lab/build/live
log       412 lines
stop      Ctrl-C
```

6.2 Why a small server is necessary

You cannot open `web/index.html` as a file and get a working display. A browser does not let a page opened from disk read other files from disk, so the page cannot reach your mission log. `make watch` starts a small web server for that reason, and for one other: the page asks each second only for the part of the log it has not read yet, and a plain file server would send the whole log every time.

6.3 If you work over SSH

`localhost` means the computer in front of you, and the server runs on the remote machine. Do one of these:

```
# on your own computer, then open http://localhost:8080/
ssh -L 8080:localhost:8080 <user>@<host>
```

```
# or publish it on the network
make watch BIND=0.0.0.0
```

The second form lets everyone on that network read your run.

6.4 What the display shows after a Lab 1 run

The display has six views. Lab 1 fills three of them, and that is correct: the others need data that Lab 2, Lab 3 and Lab 4 produce.

View	After a Lab 1 run
Mission log	complete. Every line your <code>log_wrotef</code> wrote, coloured by level
Contacts	complete. Every report, read from <code>feed.txt</code>
Processes	one entry: your demo. Its pid, its parent, its name, its open descriptors
Dashboard	the counts that follow from those two files. The worker and queue panels wait for Lab 3
Situation	the states your mission log can establish
Bench	nothing. It shows a hardware sensor board, which this course uses later

The Processes view gets the pid, the parent pid, the process name and the descriptor count from one line of your mission log: the process line, the second line in the table in Section 7.6. If that view is empty, your demo did not write that line.

The complete operations centre that this display is modelled on is shown in Figures 3 to 9 of the SENTINEL lab documentation.

7 What you build

Fifteen functions, in four files, plus a `Makefile`, your tests, and one demonstration program.

Source file	Functions
<code>src/io.c</code>	<code>sc_open_read</code> <code>sc_open_write</code> <code>sc_open_write_mode</code> <code>sc_close</code> <code>sc_write_all</code> <code>sc_read_full</code>
<code>src/clock.c</code>	<code>sc_now_ns</code> <code>sc_now_wall</code> <code>sc_stamp</code>
<code>src/log.c</code>	<code>log_open</code> <code>log_wrotef</code> <code>log_close</code> <code>log_level_name</code>
<code>src/proc.c</code>	<code>proc_report</code> <code>proc_report_write</code>

7.1 `io.c`: reading and writing without losing data

Why the centre needs it. Every report enters the system through `read`, and every log line leaves through `write`. A centre that loses part of a transfer loses part of the record, and nothing above this layer can recover it.

The raw system calls have two behaviours that look like errors and are not:

- **A partial transfer.** `write` may take fewer bytes than you offered, and `read` may return fewer than you asked for. Neither is a failure. Code that assumes one call moves everything works on small files, and then loses data the first time a call is interrupted or the descriptor is a pipe or a terminal.
- **EINTR.** A signal arrived while the call was waiting. Nothing went wrong. The call must simply be made again. Part 2 installs a signal handler, so every blocking call in SENTINEL can meet

this.

Your wrappers loop for both cases, so that no caller above them has to remember.

Call read and write directly in `sc_read_full` and `sc_write_all`. Do not use `fread`, `fwrite` or another library function in their place. The tests that mark this lab supply their own `read` and `write`, which move a few bytes at a time and fail with `EINTR`. That is how the tests prove that your loops work, and it works only when your wrappers call those two functions.

The mode of a new file passes through the **umask** of the process. The umask is a set of permission bits that the kernel removes from every file that the process creates. With the usual umask 022, a request for 0666 gives 0644. `sc_open_write` and `sc_open_write_mode` give the mode to `open` and let the umask apply. They do not change the mode afterwards.

7.2 clock.c: two clocks, and why they are different

Why the centre needs it. A report with no reliable time on it cannot be placed in a sequence of events, and a sequence of events is what an operations centre produces.

There are two clocks, and using the wrong one is the most common fault in this lab. The compiler cannot catch it.

Function	Clock	The question it answers
<code>sc_now_wall</code> , <code>sc_stamp</code>	<code>CLOCK_REALTIME</code>	What is the date and time?
<code>sc_now_ns</code>	<code>CLOCK_MONOTONIC</code>	How long did that take?

`CLOCK_REALTIME` is the real date, and it **can step**. An administrator or a time service can move it forwards or backwards while your program runs. Every timestamp in a contact report or a mission log line is a date, so it uses this clock.

`CLOCK_MONOTONIC` never steps. It is meaningless as a date, and only the difference between two readings means anything. Every duration you measure uses this clock. If you measure a duration with `CLOCK_REALTIME` and the clock steps back during the measurement, you report a negative duration.

A third clock exists and is given to you, not asked of you: `cpu_thread_ns()` in `common/cpu.h` reads `CLOCK_THREAD_CPUTIME_ID`, which counts only the time a thread spent on a processor. You do not need it in Lab 1. Lab 3 does.

7.3 log.c: the mission log

Why the centre needs it. This is R6. The mission log is the record of what the centre knew and when. It is read after the event, by people who were not there.

Each line has this format:

```
1755534061.482913 SYSTEM   SENTINEL Part 1 starting, feed=build/live/feed.txt
1755534061.483204 INFO     contact ACCF-4587 from NWS
1755534061.483411 PRIORITY priority contact ACCF-4587 from NWS at 68.7100,-105.7000 speed 420.0
```

The timestamp, then the level name in a field eight characters wide and left aligned, then one space, then the message. There are six level names: `SYSTEM`, `INFO`, `PENDING`, `PRIORITY`, `ALERT` and `TRACK`. Lab 1 uses three of them. A line is at most 512 bytes.

Open the log with `O_APPEND`. This is not a detail, and it is worth understanding now rather than in Lab 2. `O_APPEND` makes each `write` go to the current end of the file as one atomic step. In Lab 2, four sensor processes and the centre all write this one log. Without `O_APPEND` each of them holds its own idea of where the end is, and they write over each other: lines vanish, and every `write` reports success. A log that loses lines silently is worse than no log, because it is believed.

The log has mode `0600`, owner only, after every `log_open`. It is a surveillance record. `open` applies its mode argument only when it *creates* the file. A log that already exists keeps the permissions it had, and an earlier run can have left it readable by every account on the machine. So `log_open` must also set the mode of a file that already exists. `fchmod` sets the mode of an open file. If `fchmod` fails, `log_open` closes the file and fails too: a log that cannot be made private is not opened.

7.4 `proc.c`: what the kernel knows about your process

Why the centre needs it. An operations centre that runs for a watch must be able to report its own state: how much memory it holds, how many files it has open, whether it is leaking either. You read that from `/proc`, the interface through which Linux publishes what it knows about a running process.

`proc_report` reads `/proc/<pid>/status` for the process name, the parent pid, the memory sizes, the thread count and the two context switch counters, and counts the entries in `/proc/<pid>/fd` for the number of open descriptors. The comment of `proc_info_t` in `syscall.h` gives the exact key for each field.

When you count your own descriptors, do not count the one that `opendir` opened to read the directory.

7.5 Rules for all four files

Why the centre needs them. Every other part of SENTINEL calls this library. A fault in it that stops the program, prints to the watch officer's terminal, or leaks a descriptor is a fault in every part above it.

- **Every function that `syscall.h` does not declare is `static`.** A helper that is not `static` is visible to every program that links your library. If Lab 2 or Lab 3 uses the same name, the link fails, or it uses the wrong function.
- **The library does not print, and it does not exit.** Do not call `printf`, `fprintf`, `perror` or `exit` in `src/`. A function reports an error with its return value and `errno`, as `syscall.h` states, and the caller decides what to do. A library that exits stops the whole operations centre because of one bad file.
- **Examine the result of every system call.** Each call that can fail has a branch for the failure.
- **An error path cleans up, and keeps `errno`.** When a function fails part way, it closes each descriptor that it opened and frees the memory that it allocated. `close` and `free` can change `errno`, so save `errno` before the cleanup and put it back after. The caller must see the error that caused the failure.

- **A comment says why, not what.** A comment that repeats the code in words adds nothing. A comment that states a reason, or a condition that is not clear from the code, is useful.

7.6 demo_lab1: the program that puts it together

```
build/demo_lab1 <feed-file> [mission-log] [--follow IDLE_MS]
```

It opens the mission log, writes a start line, writes one line describing its own process, reads the feed one line at a time, gives each line to `contact_parse`, writes one log line for each line of the feed, and at the end prints a summary and its own process report.

This section is the complete specification of `demo_lab1`. The acceptance test runs your demo as a program and compares what it writes with the rules below, character for character. Nothing that the test checks is left out of this section.

The arguments

Argument	Rule
<feed-file>	required. The file of report lines to read
[mission-log]	optional. The log to write. With no value, the log is <code>/tmp/sentinel_mission.log</code>
--follow IDLE_MS	optional, and it can come before, between or after the other two. IDLE_MS is a whole number above 0

Every argument that is not `--follow` or its value is a file name: the first is the feed, and the second is the mission log. `--follow` takes its value from the next argument: `--follow=3000` is not that form, so it is a file name. If `--follow` is given twice, the second value counts. If the feed is missing, if `--follow` has no value or a value that is not a whole number above 0, or if there is a third file name, write a usage line to standard error and exit with status 2. Do not open the mission log in that case. The text of the usage line and of every message on standard error is your choice.

The steps

Do these steps in this order.

1. Open the mission log with `log_open`. If it fails, write the reason to standard error and exit with status 1.
2. Write the start line to the log.
3. Call `proc_report(0, &before)`. Write the process line to the log from `before`. This report is also the `before` count of the leak check. If `proc_report` fails, write the reason to standard error, close the log, and exit with status 1.
4. Open the feed with `sc_open_read`. If it fails, write the reason to standard error, close the log, and exit with status 1. The log then holds two lines.
5. Give the descriptor to `lr_open`, or to `lr_open_follow` with `--follow`, with `own_fd` set to 1. Then `lr_close` closes the feed for you. Give `bufsize` as 0 for the default, or as 256 or more. A buffer that cannot hold a report of `CONTACT_LINE_MAX` bytes refuses good reports

with `EMSGSIZE`. If the reader cannot be made, close the feed and the log, and exit with status 1.

6. Read `sc_now_ns`. The elapsed time starts here.
7. Read the feed until the end, one line at a time, and write one log line for each line. The rules are in the table below and in the next part of this section.
8. Read `sc_now_ns` again. The elapsed time ends here.
9. Call `lr_close`, then `proc_report(0, &after)`. This report is the **after** count of the leak check. If it fails, write the reason to standard error, and continue: the exit status will be 1.
10. Print the summary to standard output, then the process report with `proc_report_write`, then the `open_fds` line. If step 9 failed, print the summary only: without the *after* report, the process report and the leak check have no true value to show.
11. Write the last line to the log, and call `log_close`.
12. Exit with the status in the table at the end of this section.

With `--follow`, the idle time starts at step 6, and it starts again at each new line, including a line that the reader refuses with `EMSGSIZE`.

Why the leak check is counted at those two points. At both points, the mission log is open and the feed is closed. So the two counts are equal when every descriptor that you opened for the feed is closed. If you count *before* ahead of `log_open`, or *after* behind `log_close`, the counts differ by one, and a correct program reports a leak.

The mission log

Each line has the format of Section 7.3: the timestamp, the level, and the message. `log_writetf` adds the timestamp and the level. The table gives the level and the exact message of each line that the demo writes. Text in angle brackets is a value. Everything else is written as it is shown, with one space where the table shows a space. Each message is one line: the table wraps a long message only to fit it on the page.

When	Level	Message
first line	SYSTEM	SENTINEL Part 1 starting, feed=<feed>
second line	SYSTEM	pid=<pid> ppid=<ppid> name=<name> open_fds=<fds>
each report with flag R	INFO	contact <id> from <sensor>
each report with flag P	PRIORITY	priority contact <id> from <sensor> at <lat>,<lon> speed <speed>
each line that is not a report	INFO	rejected line <n>: <reason>
a read error	SYSTEM	feed read failed: <error>

When	Level	Message
last line	SYSTEM	Part 1 complete: <read> read, <accepted> accepted, <rejected> rejected, <priority> priority, <ms> ms

The values:

- **<feed>** is the feed argument exactly as it was given on the command line.
- **<pid>**, **<ppid>**, **<name>** and **<fds>** are pid, ppid, name and open_fds from the *before* report of step 3.
- **<id>**, **<sensor>**, **<lat>**, **<lon>** and **<speed>** come from the record that `contact_parse` filled. Write **<lat>** and **<lon>** with 4 digits after the point, and **<speed>** with 1: the `printf` formats `%.4f` and `%.1f`.
- **<n>** is the number of the rejected line in the feed. The first line of the feed is 1, and every line counts, including an empty line and a line that is not a report.
- **<reason>** is the text that `contact_strerror` gives for the error that `contact_parse` returned. For a line that is too long for the reader (`EMSGSIZE`), it is `contact_strerror(CONTACT_ERR_TOO_LONG)`, which is `line too long`. An empty line has the same reason: `contact_parse` returns `CONTACT_ERR_TOO_LONG` for a line of length 0.
- **<error>** is `strerror(errno)`.
- **<read>** counts every line of the feed, and **<accepted>** plus **<rejected>** equals it. **<priority>** counts the accepted reports with flag P. A rejected line is never counted as priority.
- **<ms>** is the elapsed time of steps 6 to 8, in milliseconds, with 3 digits after the point: `%.3f`.

Four rules follow from the table, and the acceptance test checks each one:

- Lines 3 onwards hold one line for each line of the feed, in the order of the feed. A feed of k lines therefore gives a log of $k + 3$ lines. A read error adds its own line before the last line, so the log then holds $k + 4$, where k is the lines read before the error.
- An empty feed gives exactly three lines: the start line, the process line and the last line.
- The last line of a feed can end with no `\n`. The reader from `lr_open` still returns it, and it is a line like any other. **The reader from `lr_open_follow` does not:** a writer can still be writing that line, so the reader keeps it until its `\n` arrives. If the watch ends first, that line is not a line of the feed. It is not counted and not logged.
- No log line is longer than 512 bytes, with its `\n`.

A read error is any error from `lr_next_line` other than `EMSGSIZE`, and other than `EAGAIN` with `--follow`. After a read error, stop reading and continue from step 8. A read error is not a line, so it is not counted in **<read>**. The run then exits with status 1.

If `log_wrtief` fails during the run, continue. The mission log is the record, but a centre that stops because one line could not be written also stops reading the sensors.

Your own tests can check your demo against this table in the same way, because the table is exact.

Following a feed that is still being written

`--follow` reads the feed as a stream, while something else is still appending to it. This is the one part of Lab 1 that cannot be guessed, so it is worth reading twice.

On a regular file, `read` returning 0 means “**nothing after this offset yet**”. It does not mean “nothing ever”. Another program can append, and the next `read` from the same offset returns the new bytes. This is how `tail -f` works.

The given reader knows this. `lr_open_follow` never reports the end of input. Where `lr_open`’s reader would return 0, it returns -1 with `errno` set to `EAGAIN`, and leaves the decision to you. Your loop must:

- treat `EAGAIN` as “**nothing yet**”. Wait a short time, 50 ms is enough, and ask again. A loop with no wait uses a whole processor while it waits.
- stop when no new line has arrived for `IDLE_MS` milliseconds. A loop with no idle timeout never ends.
- treat `EMSGSIZE` as **something else entirely**. It means one line was longer than the reader’s buffer and is lost, and the next call works normally. Count it as a rejected line and carry on. A `break` here turns one bad line into a dead feed: one malformed report silences that channel for the rest of the watch.

The two errors are not the same answer, and the difference matters operationally. `EAGAIN` means wait. `EMSGSIZE` means one report is lost. A centre that stops reading a sensor because of one bad line has stopped watching, and nothing tells the watch officer.

To see this behaviour, run a live watch. A live watch uses three terminals. A fourth terminal sends the satellite pass.

Start Terminal 1 first. Terminal 1 creates the feed file. Your program must have a file to open.

```
# terminal 1: three sensors, each at its own rate
cd ~/sentinel-lab
mkdir -p build/live
rm -f build/live/feed.txt build/live/missions.log
touch build/live/feed.txt
```

```
build/contactfeed --sensor NWS --rate 20 --profile world >> build/live/feed.txt &
build/contactfeed --sensor AUR --rate 5 --profile world >> build/live/feed.txt &
build/contactfeed --sensor AIS --rate 50 --profile world >> build/live/feed.txt &
wait
```

Use three sensors, and include AIS. The coastal receiver watches the strait. The contact in Section 3 is in that strait. The radar watches the Arctic, so a watch that uses the radar alone does not show that contact.

The two `rm` commands are necessary. `make lab1` writes a complete feed and a mission log, and it leaves both files in place. Your program opens the mission log with `O_APPEND`. Without these two commands, your counts include the earlier run, and the mission log holds two runs together.

```
# terminal 2: your library reads while the three sensors write
cd ~/sentinel-lab
build/demo_lab1 build/live/feed.txt build/live/missions.log --follow 3000

# terminal 3: the display
cd ~/sentinel-lab
make watch
```

Open <http://localhost:8080/> and select the Mission log view. The view fills while the watch runs.

The satellite pass is optional. Start it a few seconds after Terminal 2. It sends 2000 reports as fast as the file accepts them. Part 3 must absorb this load.

```
cd ~/sentinel-lab
build/contactfeed --sensor SAT --burst 2000 --profile world >> build/live/feed.txt
```

The kernel's view of your program

While the watch runs, the kernel keeps a description of your program in the directory `/proc/<pid>`. Your `proc_report` reads the same files once at the start and once at the end. In this step you read them each second, and you see them change.

Start this after Terminal 2, in another terminal. `pgrep` finds the pid of your running demo, and `watch` runs the command again each second:

```
pid=$(pgrep -nx demo_lab1)
watch -n 1 "grep -E '^(State|VmRSS|Threads|voluntary|nonvoluntary)' \
  /proc/$pid/status; ls -l /proc/$pid/fd; \
  head -n 2 /proc/$pid/fdinfo/3 /proc/$pid/fdinfo/4"
```

If `pgrep` prints nothing, your demo is not running: start Terminal 2 first. Press Ctrl-C to stop watch. A run with a wait of 50 ms shows this:

```
State:  S (sleeping)
VmRSS:    792 kB
Threads:    1
voluntary_ctxt_switches:    50
nonvoluntary_ctxt_switches:    0
total 0
lrwx----- 1 you you 64 Oct  9 10:00 0 -> /dev/pts/1
lrwx----- 1 you you 64 Oct  9 10:00 1 -> /dev/pts/1
lrwx----- 1 you you 64 Oct  9 10:00 2 -> /dev/pts/1
l-wx----- 1 you you 64 Oct  9 10:00 3 -> /home/you/sentinel-lab/build/live/missions.log
lr-x----- 1 you you 64 Oct  9 10:00 4 -> /home/you/sentinel-lab/build/live/feed.txt
==> /proc/12345/fdinfo/3 <==
pos:    10461
flags:  0402001

==> /proc/12345/fdinfo/4 <==
pos:    9957
flags:  0400000
```

What you see	What it shows	Where Lab 1 uses it
State: S (sleeping) for most of the run	your program sleeps in its wait, and does not use the processor	the EAGAIN rule above
voluntary_ctxt_switches rises by about 20 each second	each wait of 50 ms gives the processor up: $1000 / 50 = 20$	vol_ctxt in proc_info_t
fd 3 is missions.log, fd 4 is feed.txt	your demo opened the log first, then the feed, in the order of the steps	the steps of the demo, and open_fds
pos: of fd 4 grows, and stays near the size of the feed	the offset in the feed. It is the point to which your program has read	read returning 0 means “nothing after this offset yet”
flags: of fd 3 ends in 2001	2000 is O_APPEND, and 1 is O_WRONLY	log_open
pos: of fd 3 is the size of the log	with O_APPEND, each write goes to the end of the file	log_open

Compare the offset with the size of the feed: `stat -c %s build/live/feed.txt`. While the sensors write, the offset is a little behind the size, because new lines arrive all the time. When you stop the sensors in Terminal 1, the offset becomes equal to the size. Your program then waits for its idle time, three seconds, before it stops.

A loop with no wait shows the fault that the EAGAIN rule prevents. The state is R (running), voluntary_ctxt_switches stays at 0, and nonvoluntary_ctxt_switches rises: the program never gives the processor up, and the kernel must take it away. If your watch shows this, your loop does not wait.

When your program stops, watch shows that `/proc/<pid>` does not exist any more. The kernel removes the directory with the process. This is why `proc_report` of a pid with no process gives ENOENT.

How to stop the watch

Press Ctrl-C in Terminal 1. The three sensors stop.

Your program waits three seconds for a new line. No line arrives. Your program prints its summary and stops.

This is the behaviour the lab tests. The run must stop because the feed is quiet. It must not stop because the reader reported an error.

Press Ctrl-C in Terminal 3 to stop the display.

Faults, and what each one means

What you see	What it means
lines read 0, and your program stops after three seconds	Terminal 1 did not run. The feed file stayed empty
Your program cannot open the feed	build/live/feed.txt does not exist. Run the touch command in Terminal 1
Your program stops immediately, and the count is near 1000	The feed still holds the run from make lab1 . Run the two rm commands
Your program does not stop	Your loop has no idle timeout. This section states the rule

The standard output

```
SENTINEL Part 1 acceptance
feed          build/live/feed.txt
mission log   build/live/missions.log
lines read    1000
accepted      1000
rejected      0
priority      100
  NWS         250
  AUR         250
  AIS         250
  SAT         250
elapsed       1.508 ms
throughput    663130 reports/s
```

```
process report
pid           12345
ppid          12344
name          demo_lab1
vm_size       2456 kB
vm_rss        1204 kB
threads       1
open_fds      4
ctxt_vol      3
ctxt_invol    0
open_fds before=4 after=4 (balanced)
```

The last line is the leak check. (**balanced**) means the descriptor count at the end equals the count at the start, so every descriptor you opened was closed. (**LEAKED**) means one stayed open. A centre that leaks one descriptor for each report stops being able to open files part way through a watch.

Three parts of this output are required. The rest is an example, and you can change it.

- The four count lines **lines read**, **accepted**, **rejected** and **priority**, each with its number. They must equal the numbers on the last log line. The spaces between a name and its number

are your choice.

- The process report, written by `proc_report_write` from the *after* report of step 9.
- The leak check line, in exactly this form, with the *before* and *after* counts of `open_fds`:

```
open_fds before=<before> after=<after> (balanced)
```

with (LEAKED) in place of (balanced) when the two counts differ. The spaces at the start of the line are your choice.

Call `fflush(stdout)` before `proc_report_write`. `printf` keeps its output in a buffer inside your program, and sends it only when the buffer is full or the program ends. `proc_report_write` calls `write`, which sends its output at once. Without `fflush`, the process report appears before the summary, on the same standard output.

The exit status

The exit status tells the program that started the demo how the run ended.

Status	Meaning
0	the run finished, and at least one line was accepted or the feed was empty
1	the log or the feed could not be opened, a <code>proc_report</code> failed, a read error stopped the reading, or the feed held lines and every one was rejected
2	the arguments were wrong

Status 1 for a read error, a failed step 9 or a feed of rejected lines comes at the end of the run, not in place of it. The demo still does every step in the list above, so the log holds its last line. The status tells the caller that the record of this run is not complete, or that nothing in the feed was a report.

7.7 Your Makefile

It must do four things: build `$(BUILD)/libsyscall.a` from every file in `src/`; build `$(BUILD)/demo_lab1`; provide a `test` target that builds and runs each `tests/test_*.c` and stops at the first failure; and provide a `clean` target that removes only what it built.

The top-level Makefile passes you three variables. Set each with `?` so that your Makefile also works on its own. `make` itself gives `CC` the value `cc` before it reads your file, so `CC ?= gcc` does not change it when you run your Makefile alone. On Linux `cc` is `gcc`, and the result is the same. Keep the line: it states your default, and it is marked.

Variable	Value from the top level	Your default
CC	gcc	gcc
CFLAGS	-std=c11 -D_GNU_SOURCE -Wall -Wextra -Werror -g -O1	the same
BUILD	the absolute path of <code>build/</code>	<code>\$(abspath ../../build)</code>

Compile with `-Iinclude -I../../common`. Link each program with `$(BUILD)/libsyscall.a $(BUILD)/libcommon.a -lm`, in that order: the linker reads left to right, and takes from an archive only what is still undefined when it reaches it.

A warning stops the build. Do not remove `-Werror`.

7.8 Your tests

Write one test program for each source file: four files, four tests. Name each one after its source file: `tests/test_io.c`, `tests/test_clock.c`, `tests/test_log.c` and `tests/test_proc.c`. A test is a plain C program that uses `assert` and returns 0 when every assertion passes. Do not use a test framework.

A test decides for itself whether the code is correct. A program that only prints values for a person to read is not a test, and it is marked as one that tests almost nothing.

A test must not depend on the directory it runs in. `make test` runs your tests with `src/libsyscall/` as the working directory, and the marker can also run a test program on its own, from another directory. A test that opens `tests/data.txt` or `../build/x` then finds nothing there, and fails. Make each file that a test needs inside the test itself, at an absolute path under `/tmp`.

Test every promise the header makes, including the errors. **After each error, show that the next call still works.** A test that proves only that an error was reported is half a test, and the fault it misses is the one that matters: a function that reports an error correctly and is then unusable. One malformed report would take a sensor off the air for the rest of the watch, and every test would still pass.

How to make read and write return less. A test program can define its own `read` and `write`. The linker then uses them in place of the C library's, for your library's calls too. Your versions can move a few bytes at a time, or fail with `EINTR`, and pass the rest to the kernel with `syscall(SYS_read, ...)` and `syscall(SYS_write, ...)` from `<sys/syscall.h>`. This is how the tests that mark this lab test your loops. It is not required in your own tests.

Do not write tests for the given code in `common/`. It is not yours, and it has its own.

7.9 Which sky to run against

`make lab1` runs the feed generator described in Section 5. Its profile chooses what kind of traffic the four sensors report. Every profile writes the same line format, so your library never changes between them.

Command	What the sensors report
<code>make lab1</code>	The current watch. Four sensors with overlapping coverage, reporting the same objects from different positions. This is the default, and the operation in Section 3 is in it
<code>make lab1 PROFILE=dirty</code>	The same, and some reports arrive damaged. Lines that are truncated, over length, or carry a field that is not a number

Run both before you submit. A library that handles only clean input passes the first and fails the second. `PROFILE=dirty` makes the error paths of `contact_parse` run. Your summary must then report a `rejected` count above zero and keep reading, because a channel that stops after one bad line has stopped watching.

No profile makes EMSGSIZE happen. The longest line that `dirty` sends is 222 bytes. The reader gives `EMSGSIZE` only for a line that does not fit in the `bufsize` you gave to `lr_open`, and `contact_parse` refuses every line longer than 128 bytes first. To test your `EMSGSIZE` branch, put one line of 5000 bytes in the middle of a feed:

```
{ head -n 5 build/live/feed.txt
  head -c 5000 /dev/zero | tr '\0' X; echo
  tail -n 5 build/live/feed.txt
} > /tmp/long.txt
build/demo_lab1 /tmp/long.txt /tmp/long.log
```

The summary must show lines `read` 11 and `rejected` of at least 1. A smaller lines `read` means that your loop stopped at the long line.

7.10 The acceptance test

```
make accept1
```

This links a given object with **your** library and runs 34 checks against it, then runs your `demo_lab1` as a black box. Each check runs in its own process, so a crash in one costs that check only and the rest still report.

```
io.c
PASS  sc_open_read of a missing file gives -1 and ENOENT
FAIL  sc_write_all writes every byte when write() is cut short
      wrote 1000 of 1048576 bytes and returned 0

io.c          6 of 7    86%
clock.c       3 of 3   100%
log.c         7 of 7   100%
proc.c        6 of 6   100%
demo_lab1    11 of 11  100%
```

33 of 34 checks pass (97%).

The line for each file tells you where to work. In the example above, open `io.c` and nothing else.

Some checks make `read` and `write` return less than you asked for, and make them fail with `EINTR`, to prove that your wrappers loop. You cannot pass those by testing on a small file.

The acceptance test is a subset of the tests used for marking. A library that passes it can still lose marks in the course tests, so your own tests in Section 7.8 still matter.

7.11 Done when

- ☐ `make` builds your library and your demo with no warning.
- ☐ `make test` passes.
- ☐ `make accept1` passes all 34 checks.
- ☐ `make lab1` and `make lab1 PROFILE=dirty` both complete, and the second reports a `rejected` count above zero (Section 7.9).

- ☐ The demo prints (balanced).
- ☐ `--follow` ends because the feed went quiet, and not because the reader reported `EAGAIN`.
- ☐ `valgrind` reports no leak and no error:

```
valgrind --leak-check=full --error-exitcode=1 \
    build/demo_lab1 build/live/feed.txt /tmp/check.log
```

Under `valgrind`, `open_fds` is higher than in a normal run, because `valgrind` opens descriptors of its own inside your process. The two counts must still be equal.

8 Assessment

Lab 1 is 100 marks in total, and it carries 5% of the course grade. Two components carry those marks.

Component	Marks	Of the course	When	Format
Deliverable	80	4%	Brightspace, Oct 23	the zip file from Section 9
Oral exam	20	1%	in the Lab 2 session, at your own machine	five minutes: a demonstration, then questions
Total	100	5%		

8.1 The prelab

The prelab is **required**, and it carries **no marks**. The paper is `docs/lab1/prelab1.pdf` in your repository, and it is also on Brightspace. You submit your answers on Brightspace by midnight on Wednesday, October 7, 2026.

It has five questions about the system calls that this lab uses. The prelab is where you prepare, and the marks come later. The oral exam draws on it: the examiner can ask you any question from the prelab. Three of the five questions describe code that you then write for the deliverable.

8.2 The oral exam

When. In the Lab 2 session, not the Lab 1 session. You have finished and submitted Lab 1 by then, so you demonstrate work that is complete. You are told the order in advance.

Where. At your own machine, with your submitted Lab 1 code on the screen.

How long. Five minutes for each student.

What happens. Three things, in this order.

- You demonstrate Lab 1.** You run your own library and show that it works: the feed read, the mission log written, the process report printed, the descriptor count balanced. This is your work, running, on your machine.
- You answer questions about your implementation.** The examiner asks about decisions you made in the code on the screen, and about what your program will do under a condition

you have not tried. A question may ask you to predict what a command will print before it is run, or what would happen if one line of your code were different.

3. **You answer questions on the theory behind the lab.** How the operating system behaves, and why the library is built the way it is: what `read` returns and when, what `O_APPEND` guarantees, which clock measures what, what `/proc` holds. Everything asked is something this lab uses. **The prelab is one source of these questions.** The examiner can ask you any question from it.

The questions. Four in total: two on the theory, one prediction about your code, and one change to one line of your code, where you state what the change does and why.

What it is testing. Whether the code you submitted is code you understand. Describing what a program does is one ability; saying what it will do under a condition you have not met before is another, and only the second one needs a model of what the operating system is doing underneath. The demonstration shows that your work runs. The questions show that you know why it runs.

How to prepare. Build the lab yourself, and keep a note of each fault you met and what caused it. `docs/DEFECTS.md` is in the repository for that. A student who has debugged their own code has already answered most of these questions once.

8.3 How the deliverable is marked

Component	Marks	Share
Acceptance test, <code>make accept1</code>	16	20%
Course tests, run against your library	16	20%
Your own tests and edge cases	12	15%
Code style and error handling	12	15%
<code>Makefile</code> and <code>valgrind</code>	12	15%
Agreement of the three submitted files	12	15%
Total	80	100%

The first two are proportional to the number of checks and test programs that pass. The course tests are not given to you. Like the acceptance test, they check only what `syscall.h` and this manual state.

The other four are assessed against the requirements that this manual states, in these sections:

Component	The requirements are in
Your own tests	Section 7.8
Code style and error handling	Section 7.5, and the three rules of the <code>--follow</code> loop in Section 7.6
<code>Makefile</code> and <code>valgrind</code>	Section 7.7, and the <code>valgrind</code> item in Section 7.11
Agreement of the three files	Section 9

9 Submission

You submit **one zip file on Brightspace**, by Friday, October 23, 2026. There are three steps.

9.1 Step 1: produce the three files of one run

Three files go in `docs/lab1/`, and all three must come from **one** run of your demo.

File	What it holds
<code>demo_output.txt</code>	the standard output of the run
<code>missions.log</code>	the mission log of the run
<code>proc_capture.txt</code>	<code>/proc/<pid>/status</code> and <code>ls -l /proc/<pid>/fd</code> , taken while it runs

The demo must run long enough for you to take the capture, so use a slow feed. Remove an old log first, because a new run adds its lines to the end of the file.

```
rm -f docs/lab1/missions.log
build/demo_lab1 <(build/contactfeed --sensor NWS --rate 20 --count 400 \
    --profile world) docs/lab1/missions.log > docs/lab1/demo_output.txt
```

That run takes about 20 seconds. While it runs, take the capture from a second terminal:

```
pid=$(pgrep -x demo_lab1)
{ grep -E '^(Name|Pid|PPid|Threads|VmRSS)' /proc/$pid/status
  echo
  ls -l /proc/$pid/fd
} > docs/lab1/proc_capture.txt
```

Run both commands in `bash`. The form `<(...)` is a feature of `bash`, and `sh` does not have it. Do not run another `demo_lab1` at the same time, or `pgrep` finds more than one pid.

`<(...)` gives your demo a pipe in place of a file, and it has three effects that are correct and expected:

- The start line of the log shows a feed name such as `/dev/fd/63`. That is the name `bash` gives the pipe.
- The process line shows `open_fds=5`, not 4. `bash` opens the pipe as descriptor 63 before your demo starts, so your demo holds it too.
- The capture lists descriptors 0 to 4 and 63. Descriptors 0 and 2 are your terminal, because the command redirects only standard output. Descriptor 1 is `demo_output.txt`, 3 is the mission log, and 4 is the feed that your demo opened. 4 and 63 are the same pipe.

Check two things before you continue.

- **Name:** in the capture must read `demo_lab1`. If it reads `bash`, you captured your shell and not the demo.
- The pid in the capture must be the pid on the second line of your mission log.

The three files are compared with each other and with your code. Three files from three different runs do not agree, and that is what the comparison finds.

9.2 Step 2: build the zip

```
make submit                # named from your login name
make submit NAME=<your name> # or name it yourself
```

`make submit` builds your code **from clean**, runs your tests, runs the acceptance test, checks that the three files above exist, and only then writes the zip.

It builds from clean deliberately. The most common reason a submission does not compile for the marker is a source file that was never added to your **Makefile**: it builds on your machine, where the object file is still in `build/` from an earlier run, and it builds nowhere else. This finds that before you upload, not after.

The zip holds your source and the given files. It does not hold `build/`, because your code is compiled again from the files in the zip.

9.3 Step 3: upload it to Brightspace

Keep the given files as the course sent them. Your submitted tree is compared with the released one, and a changed given file is a question you will be asked to answer.

9.4 A note on docs/DEFECTS.md

The repository contains `docs/DEFECTS.md`, a record of faults you met: what you saw, what caused it, and how you found the cause. It carries no marks and it is not required.

Keep it anyway. Two of the four oral exam questions, the prediction and the one-line change, are about what your own code does, and a written record of the faults you have already met and fixed is the fastest preparation there is.